

Day 15 — Production Jenkins

Status: ■ Not started

Objectives

- Harden Jenkins for production use: security, RBAC, backups
- Understand scaling strategies (static agents vs dynamic/cloud agents)
- Introduce Shared Libraries for reusable pipeline code
- Plan for high availability and disaster recovery

Lab Steps

Security hardening

- Enable Role-based Authorization Strategy, define roles (admin, developer, viewer)
- Enable CSRF protection (enabled by default in modern Jenkins — verify)
- Restrict "Script Approval" for Groovy sandbox usage
- Put Jenkins behind a reverse proxy (Nginx/Traefik) with TLS

Backups

- Install the thinBackup plugin or script a JENKINS_HOME tarball backup
- Test a full restore into a fresh instance

Scaling agents

- Compare static SSH agents vs dynamic Docker agents vs Kubernetes plugin agents
- Set executor limits per node appropriately (avoid overloading the controller)

Shared Libraries

- Create a vars/ shared library with a reusable standardPipeline() step
- Reference it from a Jenkinsfile with @Library('my-shared-library') _

Monitoring & alerting

- Install the Prometheus plugin to expose Jenkins metrics
- Configure build failure notifications (Slack/email)

Disaster recovery drill

- Simulate a controller failure, restore from backup, verify jobs and history are intact

Key Takeaways

- Never run builds directly on the controller in production — use agents
- RBAC + credential scoping limits blast radius of a compromised job

- Shared libraries stop teams from copy-pasting Jenkinsfile boilerplate
- Backups are only as good as your last successful restore test

Checklist

- ☐ RBAC configured with least-privilege roles
- ☐ Automated backup in place and restore tested
- ☐ At least one dynamic/cloud agent type configured
- ☐ Shared library created and used by a pipeline
- ☐ Build notifications wired up

Day 15 — Interview Questions: Production Jenkins

How do you design role-based access control (RBAC) for a multi-team Jenkins instance?

What's Jenkins's story on high availability? Since there's no native multi-controller clustering, how do teams mitigate that gap?

What should a production backup strategy for Jenkins include beyond just `config.xml`?

How would you scale build capacity — static agents vs dynamic cloud/Kubernetes agents — and when would you choose each?

What is a Shared Library, and how does it improve governance/consistency across teams' pipelines?

How do you scope credentials so one team's pipeline can't access another team's secrets?

What metrics would you monitor to catch a Jenkins controller under strain before it becomes an incident?

Describe how you'd run a disaster recovery drill for Jenkins, and what "done" looks like.

What's the risk of leaving the Groovy sandbox unrestricted, and how do you manage script approvals safely?

How do you keep plugins up to date without breaking production pipelines?

<details><summary>Answer notes</summary>

- Q2: Mitigate via fast, tested backup/restore (treat RTO as the real HA metric), durable/replicated storage for `JENKINS_HOME`, and infrastructure-as-code to rebuild the controller quickly.
- Q6: Use folder-scoped credential stores instead of one Global store; combine with RBAC so teams can only see/use credentials within their folder.
- Q10: Stage plugin updates in a non-production instance first, update in batches with a rollback plan (snapshot beforehand), and monitor for deprecation warnings in logs.

</details>